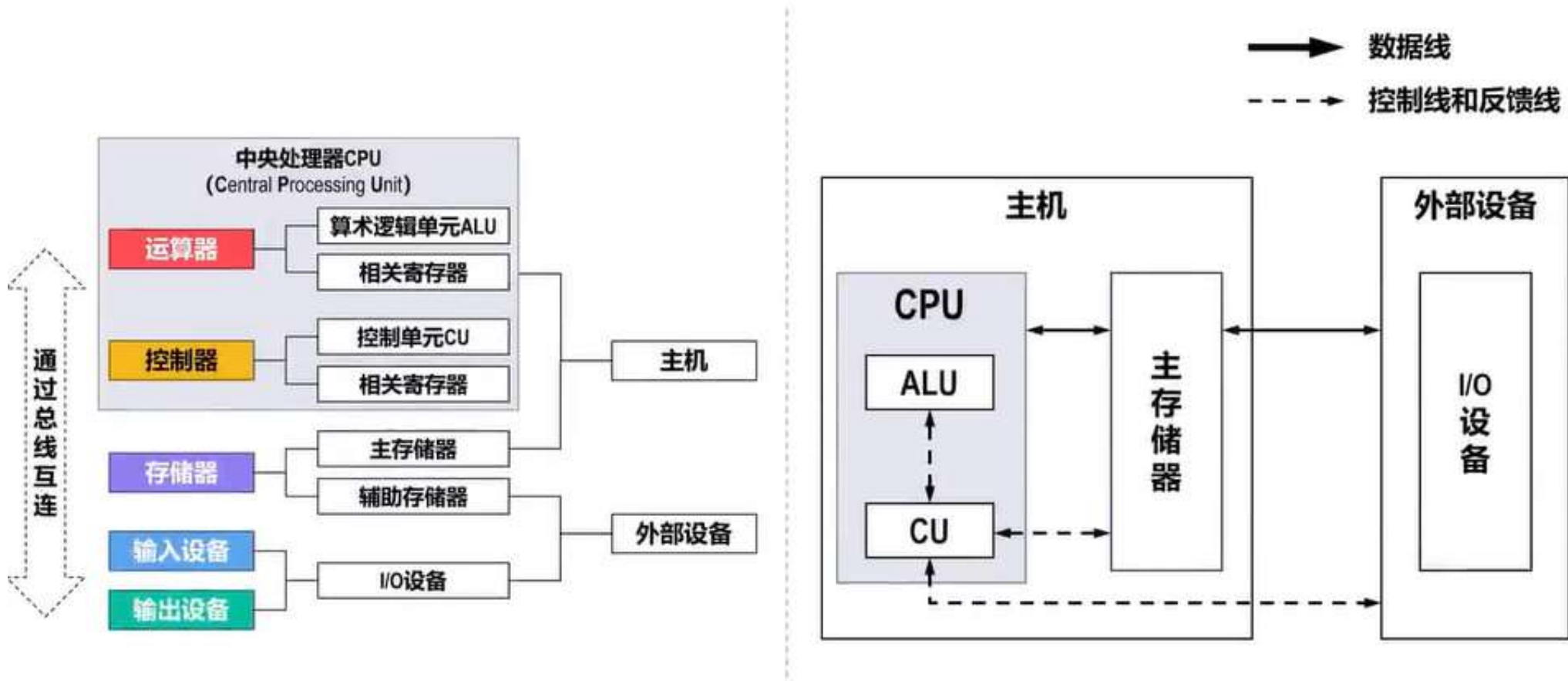


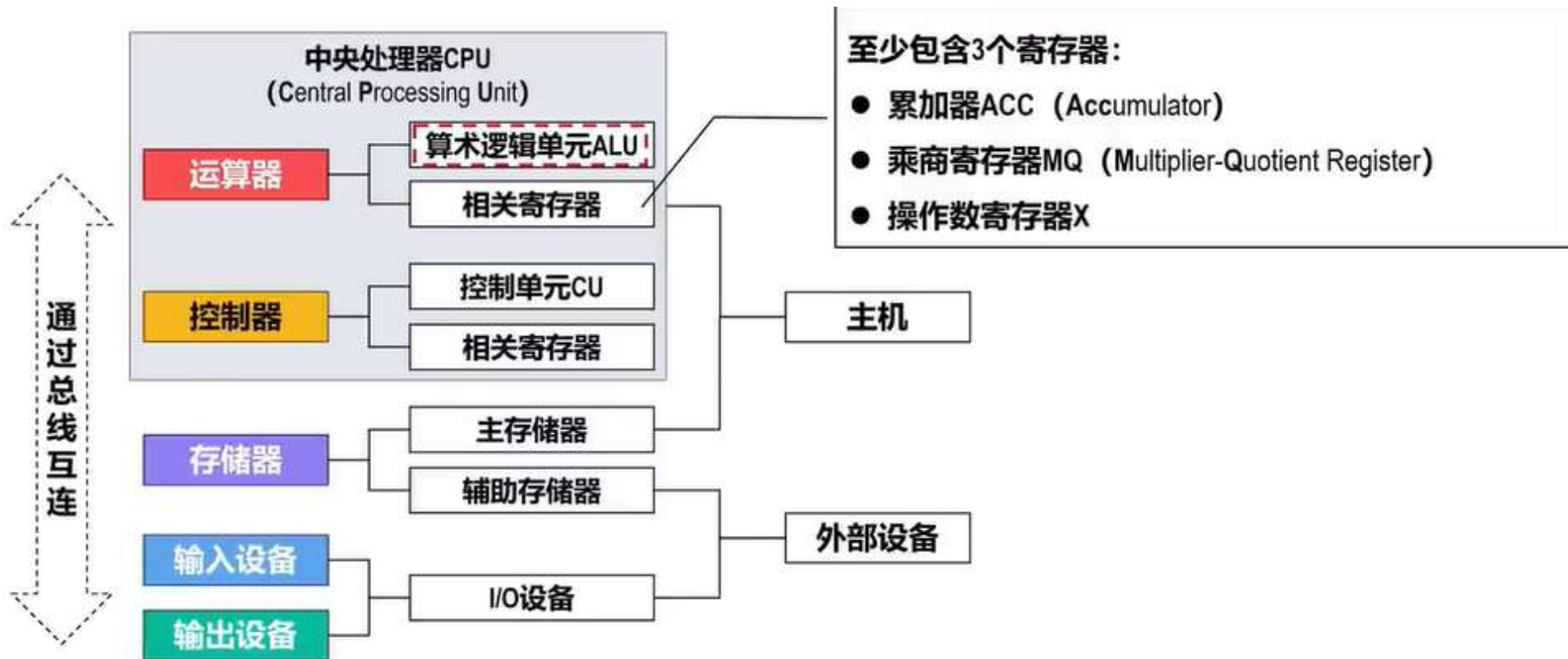
考点五：计算机运行原理

计算机硬件组成



考点五：计算机运行原理

计算机硬件组成——运算器



考点五：计算机运行原理

计算机硬件组成——运算器

ALU	加	减	乘	除
ACC	被加数 和	被减数 差	乘积高位	被除数 余数
MQ			乘数 乘积低位	商
X	加数	减数	被乘数	除数

M 表示主存储器中某个存储单元的地址
(M)表示地址为M的存储单元中的内容

ACC 表示累加器
(ACC)表示累加器中的内容

MQ 表示乘商寄存器
(MQ)表示乘商寄存器中的内容

X 表示操作数寄存器
(X)表示操作数寄存器中的内容

假设累加器ACC中已存有前一时刻的运算结果，并作为下述运算中的一个操作数。

加法操作过程

- (M)→ X

(ACC) + (X) → ACC
- 取出存放在主存中地址为M的存储单元中的内容(M)（加数），送到操作数寄存器X中
- 将累加器ACC中的内容(ACC)（被加数）与操作数寄存器X中的内容(X)（加数）相加，结果（和）保留在累加器ACC中

考点五：计算机运行原理

计算机硬件组成——运算器

ALU	加	减	乘	除
ACC	被加数 和	被减数 差	乘积高位	被除数 余数
MQ			乘数 乘积低位	商
X	加数	减数	被乘数	除数

M 表示主存储器中某个存储单元的地址
(M)表示地址为M的存储单元中的内容

ACC 表示累加器
(ACC)表示累加器中的内容

MQ 表示乘商寄存器
(MQ)表示乘商寄存器中的内容

X 表示操作数寄存器
(X)表示操作数寄存器中的内容

假设累加器ACC中已存有前一时刻的运算结果，并作为下述运算中的一个操作数。

乘法操作过程

(M) → MQ

取出存放在主存中地址为M的存储单元中的内容(M)（乘数），送到乘商寄存器MQ中

(ACC) → X

将累加器ACC中的内容(ACC)（被乘数），送到操作数寄存器X中

(X) × (MQ) → ACC // MQ

将操作数寄存器X中的内容(X)（被乘数）与乘商寄存器MQ中的内容(MQ)（乘数）相乘，结果（积）的高位保留在累加器ACC中，低位保留在乘商寄存器MQ中

考点五：计算机运行原理

计算机硬件组成——运算器

ALU	加	减	乘	除
ACC	被加数 和	被减数 差	乘积高位	被除数 余数
MQ			乘数 乘积低位	商
X	加数	减数	被乘数	除数

M 表示主存储器中某个存储单元的地址
(M)表示地址为M的存储单元中的内容

ACC 表示累加器
(ACC)表示累加器中的内容

MQ 表示乘商寄存器
(MQ)表示乘商寄存器中的内容

X 表示操作数寄存器
(X)表示操作数寄存器中的内容

假设累加器ACC中已存有前一时刻的运算结果，并作为下述运算中的一个操作数。

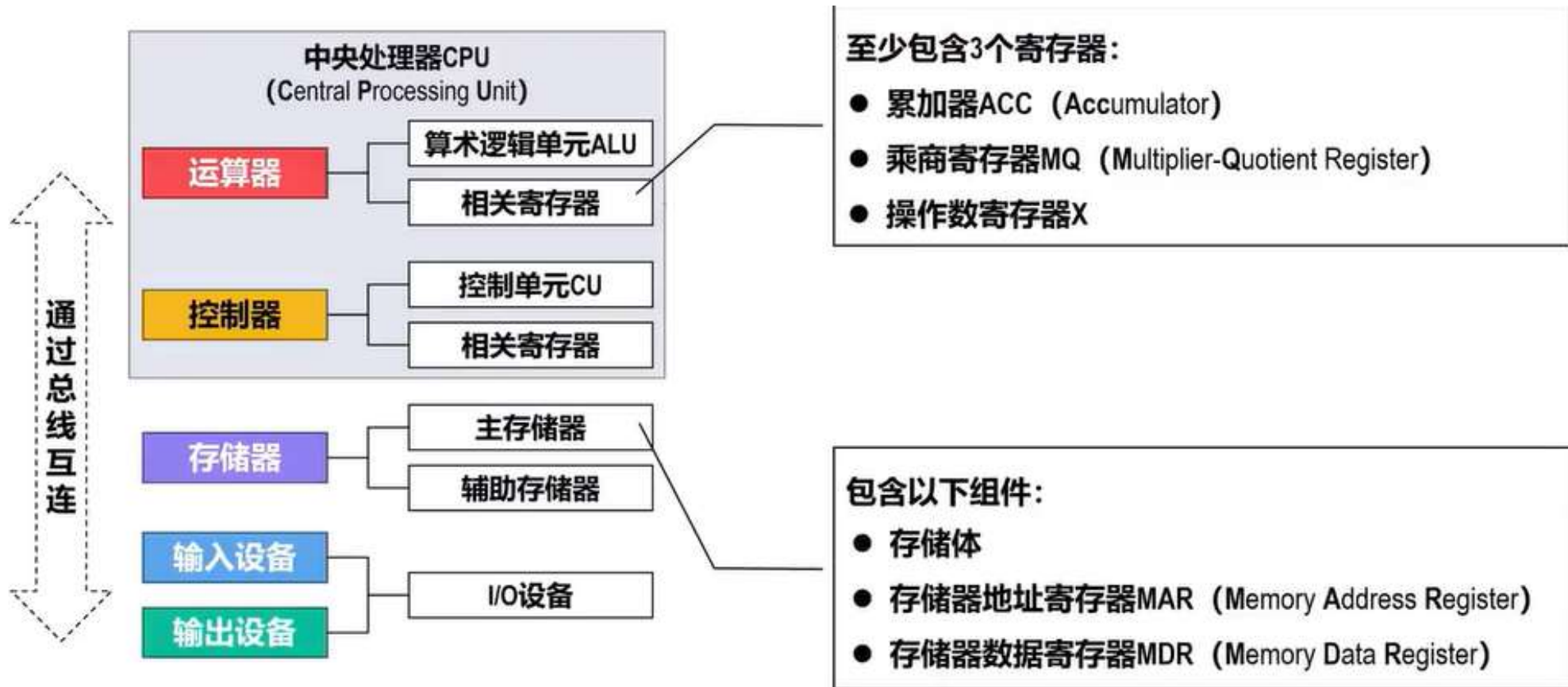
除法操作过程

- (M) → X

(ACC) ÷ (X) → MQ
- 取出存放在主存中地址为M的存储单元中的内容(M)（除数），送到操作数寄存器X中
- 将累加器ACC中的内容(ACC)（被除数）除以操作数寄存器X中的内容(X)（除数），结果（商）保留在乘商寄存器MQ中，余数保留在累加器ACC中。

考点五：计算机运行原理

计算机硬件组成——存储器



考点五：计算机运行原理

计算机硬件组成——存储器

■ 存储体由很多个存储单元组成

☐ 每个存储单元由若干个存储元件组成

◇ 每个存储元件能存储一位二进制数“0”或“1”

☐ 一个存储单元中可存储一串二进制信息，称这串二进制信息为一个存储字，这串二进制信息的位数称为存储字长（可以是8位、16位或32位等）。

■ 给每个存储单元都赋予一个编号，称为存储单元的地址。

■ 存储器地址寄存器MAR，用来存放欲访问的存储单元的地址。

☐ MAR的位数（长度），决定了存储单元的数量。

■ 存储器数据寄存器MDR，用来存放从存储体的某个存储单元取出的信息或者准备往某个存储单元存入的信息。

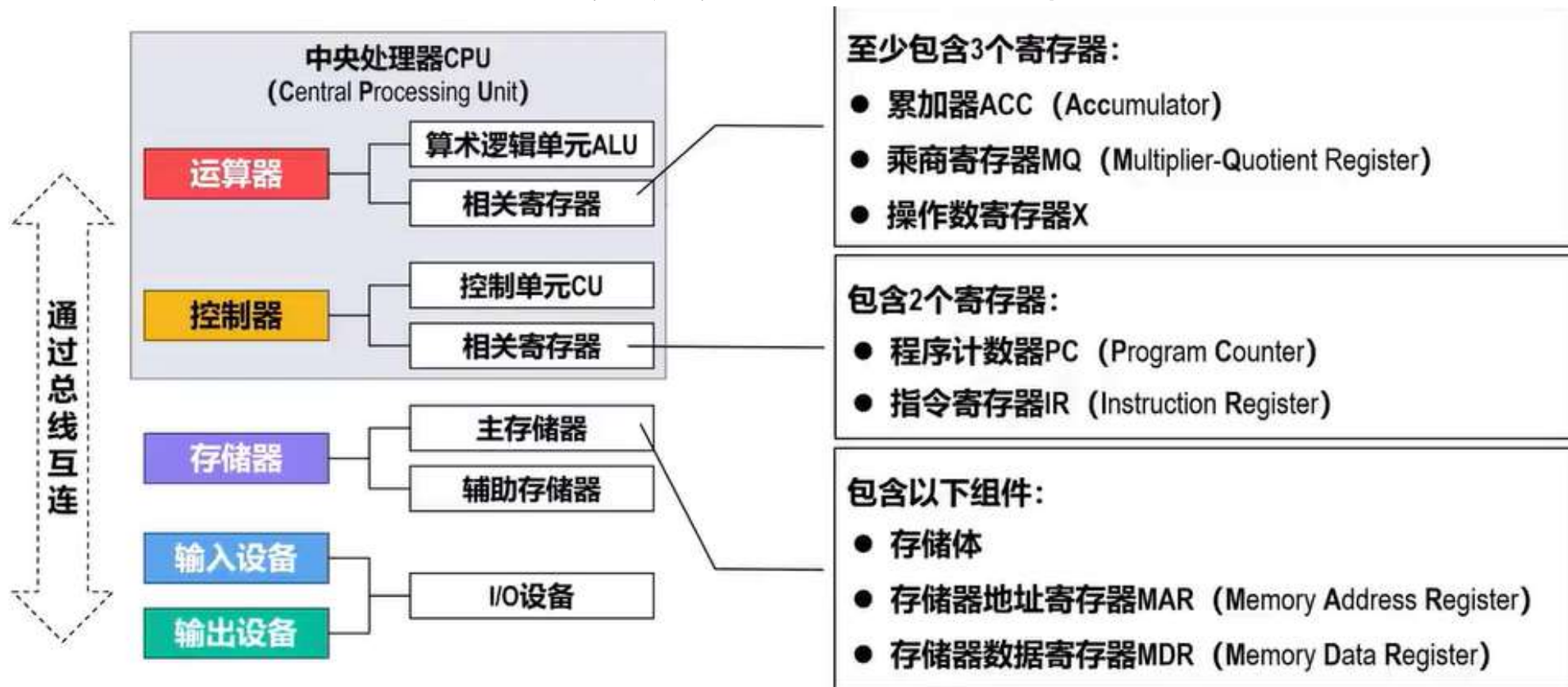
☐ MDR的位数（长度），与存储字长相等。

主存地址 (内存地址)	存储体	
0000	1 1 0 ... 1 0 1	存储单元
0001	0 1 1 ... 1 1 0	存储单元
0002	0 1 0 ... 1 0 0	存储单元
⋮	⋮	⋮
FFFE	0 0 0 ... 0 0 1	存储单元
FFFF	1 1 0 ... 0 1 1	存储单元

主存（内存）的这种按存储单元的地址来实现对其写入和读取的存取操作，需要在CPU中的控制器的控制下进行。

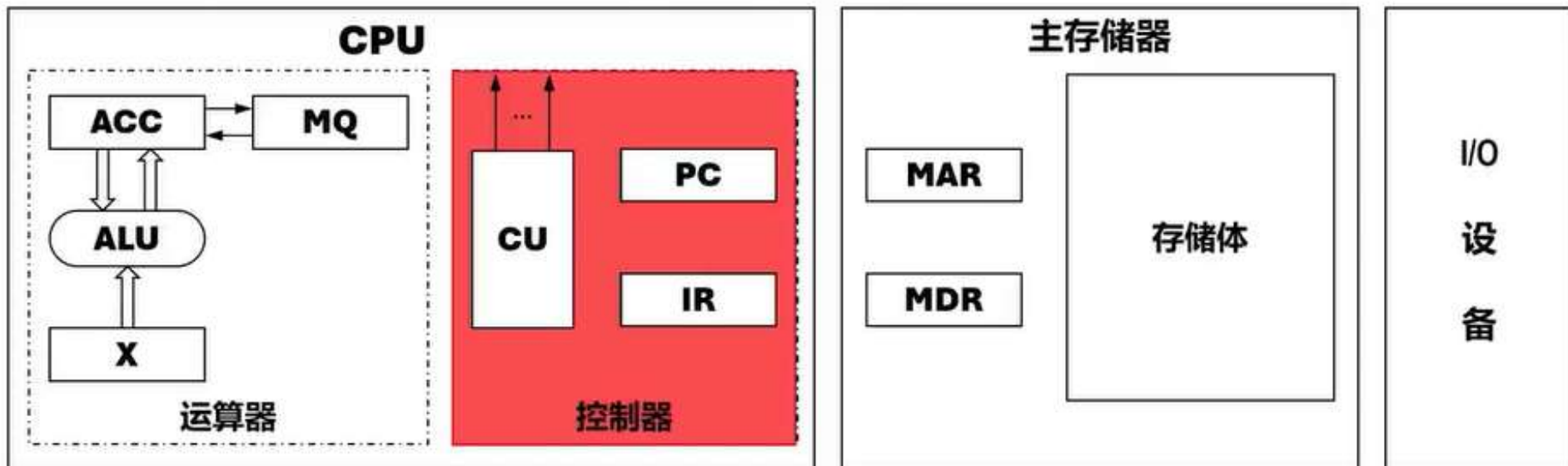
考点五：计算机运行原理

计算机硬件组成——控制器



考点五：计算机运行原理

计算机硬件组成——控制器

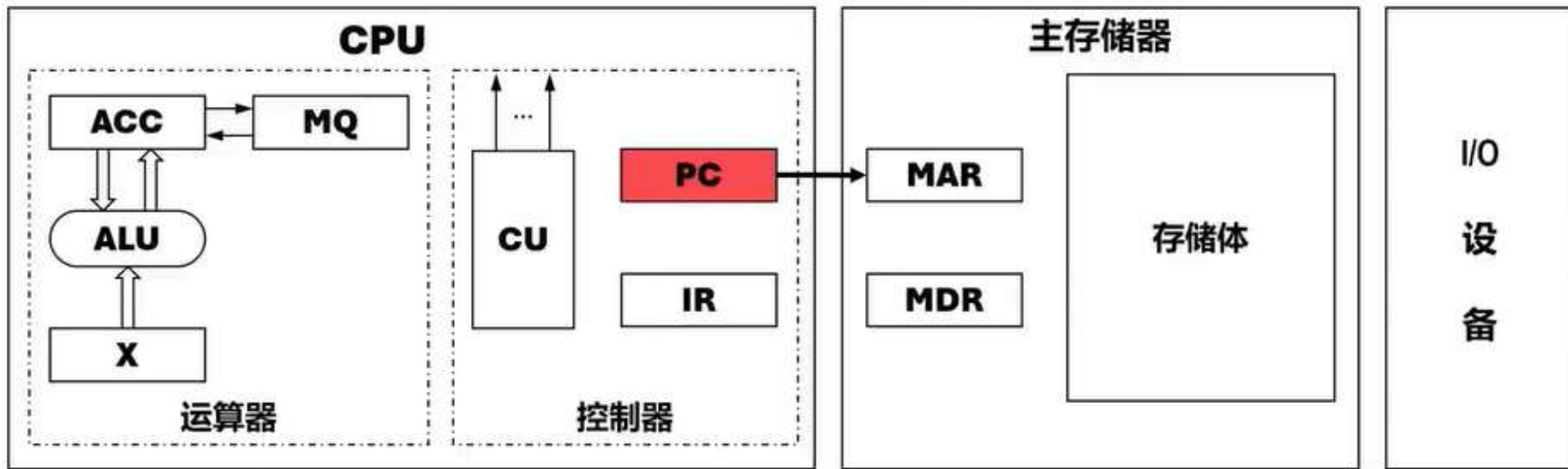


■ 控制器是计算机的神经中枢，由它指挥各部件自动、协调地工作。

- ① 控制从主存中读取一条指令，称为**取指**过程（阶段）。
- ② 对指令进行分析，指出该指令要完成何种操作，并按寻址特征指明操作数的地址，称为**分析**过程（阶段）。
- ③ 根据指令的操作码和操作数所在的地址完成某种操作，称为**执行**过程（阶段）。

考点五：计算机运行原理

计算机硬件组成——控制器



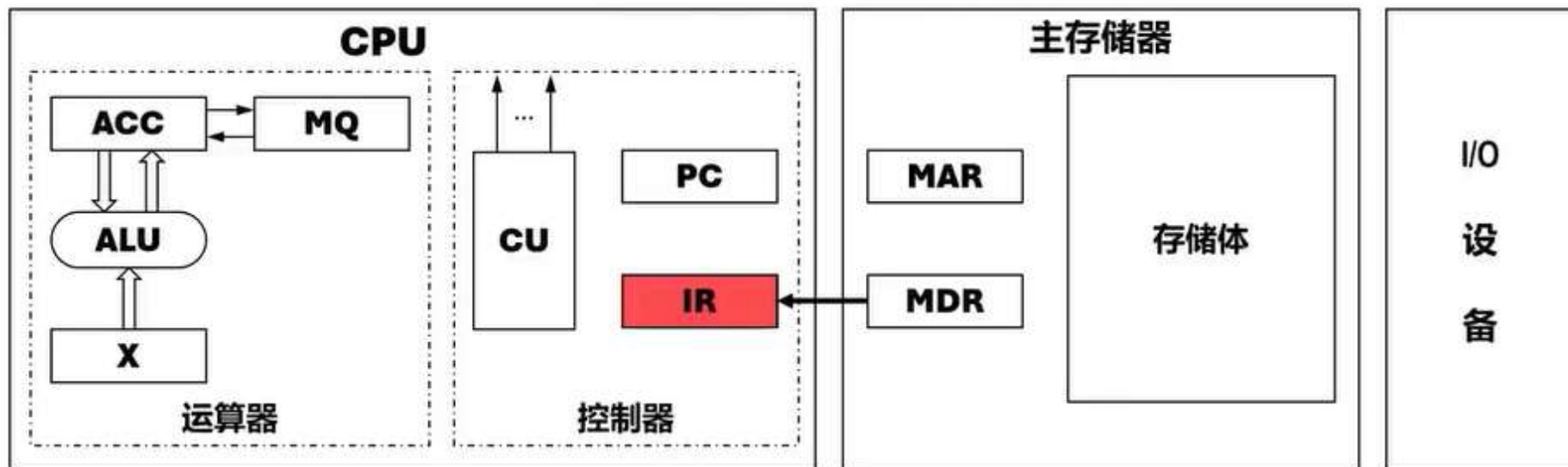
■ 程序计数器PC用来存放当前欲执行指令的地址

☐ PC与MAR之间有一条直接通路。

☐ PC自动形成下一条指令的地址（“自动加1”功能）

考点五：计算机运行原理

计算机硬件组成——控制器



■ 指令寄存器IR用来存放当前的指令

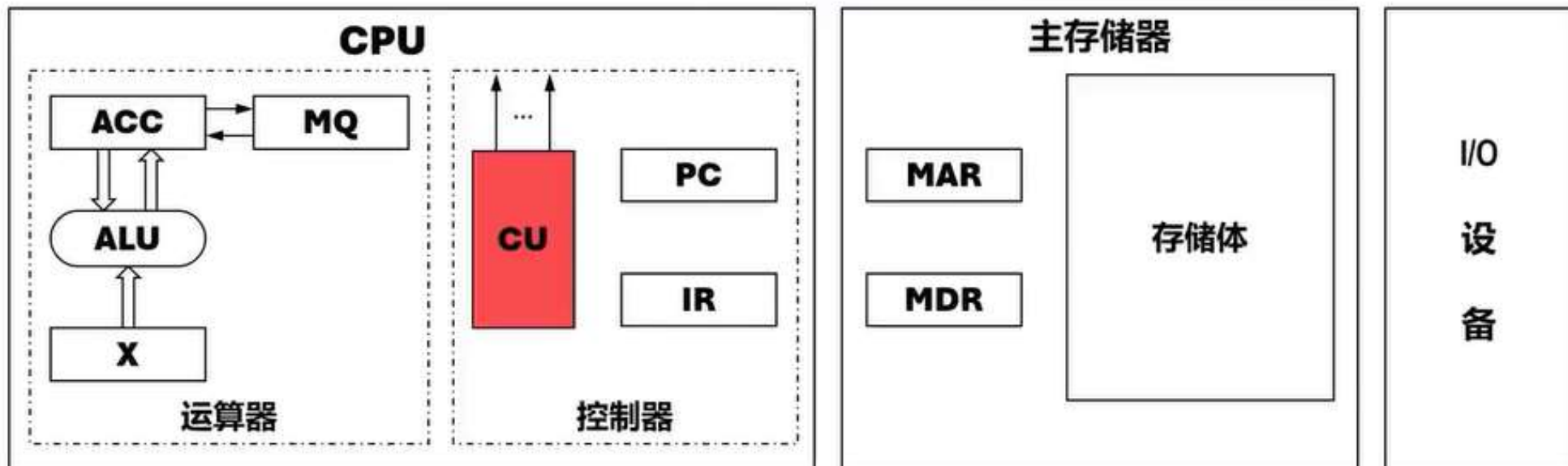
☐ IR的内容来自MDR。

☐ IR中的操作码 (用 $OP(IR)$ 表示) 会送至CU (用 $OP(IR) \rightarrow CU$ 表示), 用来分析指令。

☐ IR中的地址码 (用 $Ad(IR)$ 表示) 作为操作数的地址送至MAR (用 $Ad(IR) \rightarrow MAR$ 表示), 用来从内存中取操作数。

考点五：计算机运行原理

计算机硬件组成——控制器



■ 控制单元**CU**用来分析当前指令所需完成的操作，并发出各种微操作命令序列，用以控制所有被控对象。

考点五：计算机运行原理

计算机指令



操作码	操 作	描 述
000001	取数	将地址码所指示的存储单元中的数取到ACC
000010	存数	将ACC中的数存入地址码所指示的存储单元中
000011	加	将ACC中的数与地址码所指示的存储单元中的数相加，结果存于ACC中
000100	乘	将ACC中的数与地址码所指示的存储单元中的数相乘，结果存于ACC中
000110	停机	

考点五：计算机运行原理

计算机指令

操作码	操 作	描 述
000001	取数	将地址码所指示的存储单元中的数取到ACC
000010	存数	将ACC中的数存入地址码所指示的存储单元中
000011	加	将ACC中的数与地址码所指示的存储单元中的数相加，结果存于ACC中
000100	乘	将ACC中的数与地址码所指示的存储单元中的数相乘，结果存于ACC中
000110	停机	

编写计算 $a \times b + c$ 的机器指令程序

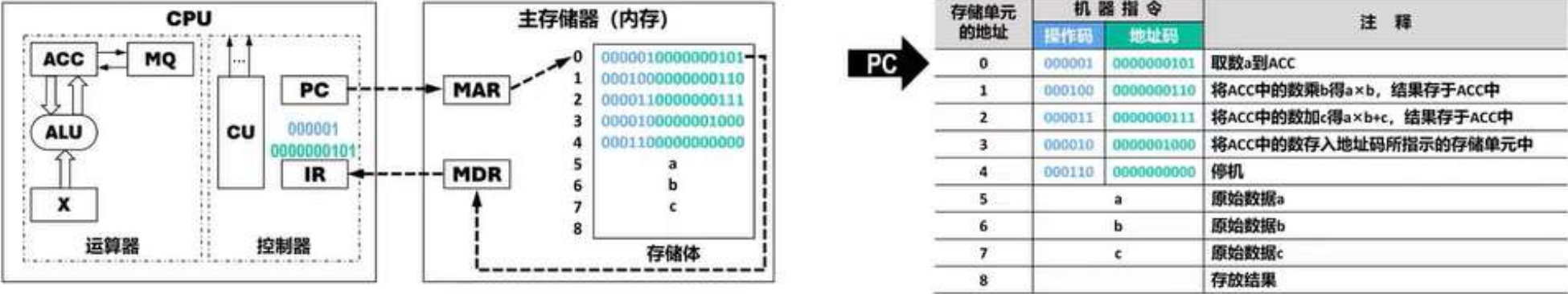


存储单元 的地址	机 器 指 令		注 释
	操作码	地址码	
0	000001	0000000101	取数a到ACC
1	000100	0000000110	将ACC中的数乘b得 $a \times b$ ，结果存于ACC中
2	000011	0000000111	将ACC中的数加c得 $a \times b + c$ ，结果存于ACC中
3	000010	0000001000	将ACC中的数存入地址码所指示的存储单元中
4	000110	0000000000	停机
5	a		原始数据a
6	b		原始数据b
7	c		原始数据c
8	$a \times b + c$		存放结果

5条
机器指令

考点五：计算机运行原理

计算机运行原理



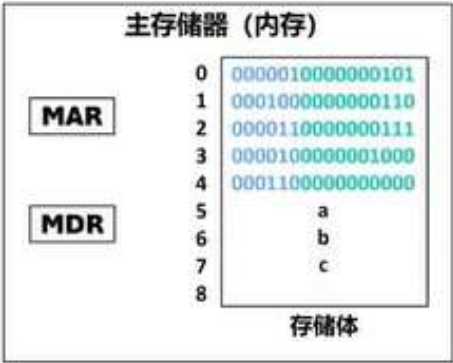
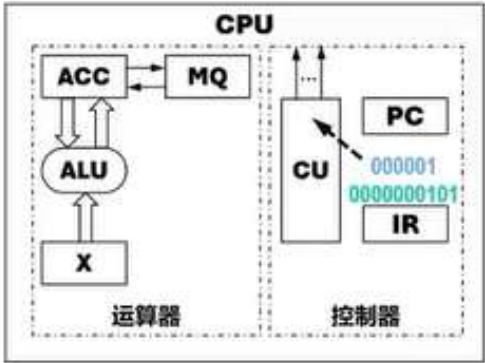
初始化: (PC)=0, 指向内存中的编号为0的存储单元, 该存储单元的内容是第一条机器指令

对第一条指令进行取指:

- ① 控制器将PC的内容送至内存的MAR (用(PC) → MAR表示) 对内存进行寻址, 并命令内存做读操作, 此时所寻址的存储单元 (0号存储单元) 的内容 “0000010000000101” 被送入MDR内。
- ② 将MDR的内容传送至控制器的IR (用(MDR) → IR表示) 。

考点五：计算机运行原理

计算机运行原理



存储单元的 地址	机 器 指 令		注 释
	操作码	地址码	
0	000001	0000000101	取数a到ACC
1	000100	0000000110	将ACC中的数乘b得 $a \times b$ ，结果存于ACC中
2	000011	0000000111	将ACC中的数加c得 $a \times b + c$ ，结果存于ACC中
3	000010	0000001000	将ACC中的数存入地址码所指示的存储单元中
4	000110	0000000000	停机
5	a		原始数据a
6	b		原始数据b
7	c		原始数据c
8			存放结果

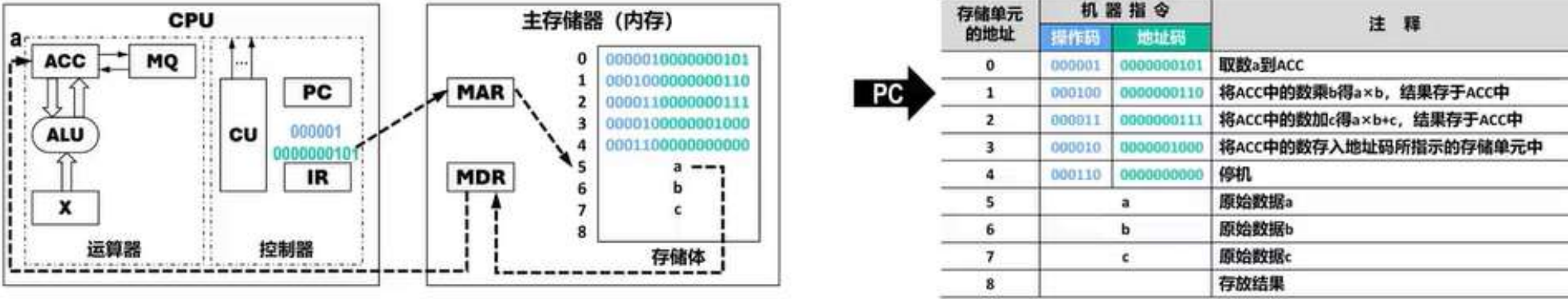
初始化: (PC)=0，指向内存中的编号为0的存储单元，该存储单元的内容是第一条机器指令

对第一条指令进行分析:

- ❶ 将IR中保存的指令的操作码送至CU进行分析（用OP(IR) → CU），操作码“000001”为取数指令。

考点五：计算机运行原理

计算机运行原理



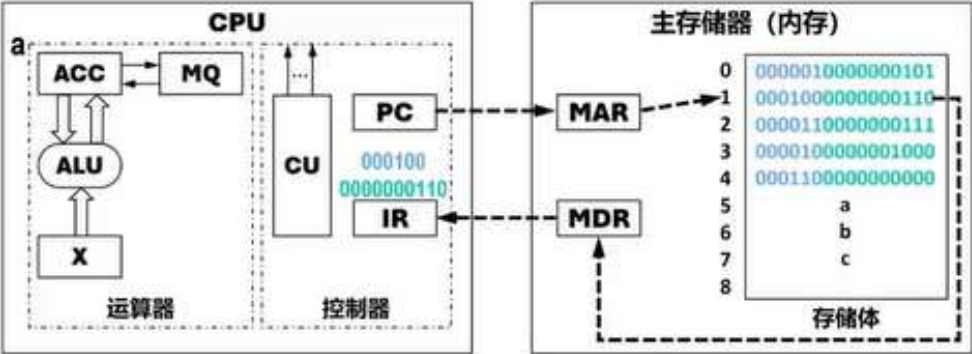
初始化: (PC)=0, 指向内存中的编号为0的存储单元, 该存储单元的内容是第一条机器指令

对第一条指令进行**执行**:

- ❶ 将IR中保存的指令的地址码送至MAR (用 $Ad(IR) \rightarrow MAR$) 对内存进行寻址, 并命令内存做读操作, 此时所寻址的存储单元 (“000000101” 号存储单元, 即5号存储单元) 的内容 “a” 被送入MDR内。
- ❷ 将MDR的内容传送至运算器的ACC (用 $(MDR) \rightarrow ACC$ 表示) 。

考点五：计算机运行原理

计算机运行原理



存储单元 的地址	机 器 指 令		注 释
	操作码	地址码	
0	000001	0000000101	取数a到ACC
1	000100	0000000110	将ACC中的数乘b得 $a \times b$, 结果存于ACC中
2	000011	0000000111	将ACC中的数加c得 $a \times b + c$, 结果存于ACC中
3	000010	0000001000	将ACC中的数存入地址码所指示的存储单元中
4	000110	0000000000	停机
5		a	原始数据a
6		b	原始数据b
7		c	原始数据c
8			存放结果

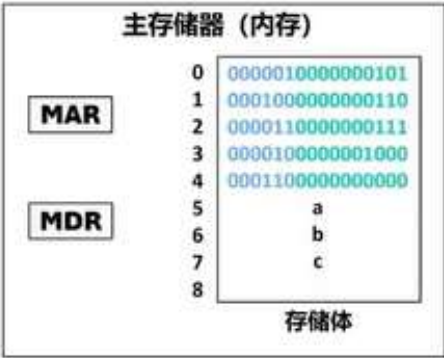
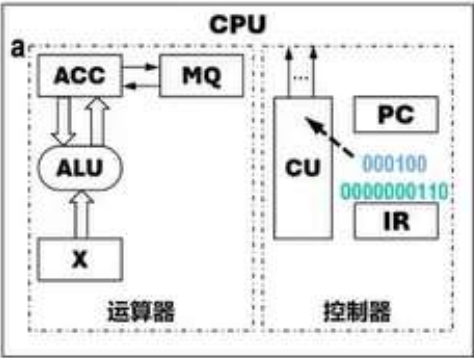
(PC)=1, 指向内存中的编号为1的存储单元, 该存储单元的内容是第二条机器指令

对第二条指令进行**取指**:

- ① 控制器将PC的内容送至内存的MAR (用(PC) → MAR表示) 对内存进行寻址, 并命令内存做读操作, 此时所寻址的存储单元 (1号存储单元) 的内容 “0001000000000110” 被送入MDR内。
- ② 将MDR的内容传送至控制器的IR (用(MDR) → IR表示) 。

考点五：计算机运行原理

计算机运行原理



存储单元的 地址	机 器 指 令		注 释
	操作码	地址码	
0	000001	0000000101	取数a到ACC
1	000100	0000000110	将ACC中的数乘b得 $a \times b$ ，结果存于ACC中
2	000011	0000000111	将ACC中的数加c得 $a \times b + c$ ，结果存于ACC中
3	000010	0000001000	将ACC中的数存入地址码所指示的存储单元中
4	000110	0000000000	停机
5	a		原始数据a
6	b		原始数据b
7	c		原始数据c
8			存放结果

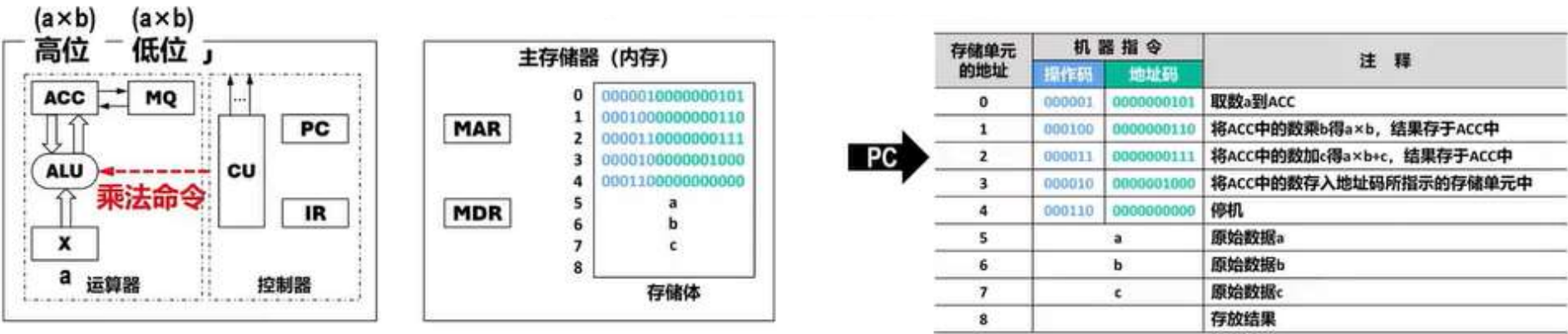
(PC)=1，指向内存中的编号为1的存储单元，该存储单元的内容是第二条机器指令

对第二条指令进行**分析**：

- ❶ 将IR中保存的指令的操作码送至CU进行分析（用OP(IR) → CU），操作码“000100”为乘法指令。

考点五：计算机运行原理

计算机运行原理



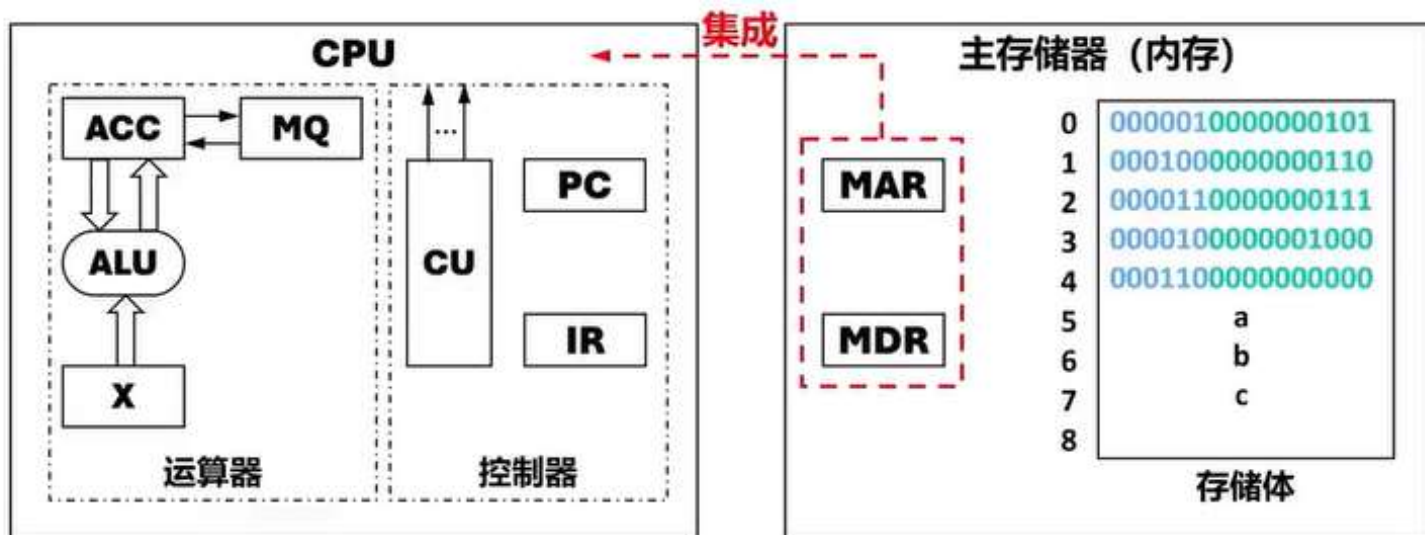
(PC)=1, 指向内存中的编号为1的存储单元, 该存储单元的内容是第二条机器指令

对第二条指令进行**执行**:

- ❶ 将IR中保存的指令的地址码送至MAR (用 $Ad(IR) \rightarrow MAR$) 对内存进行寻址, 并命令内存做读操作, 此时所寻址的存储单元 (“0000000110” 号存储单元, 即6号存储单元) 的内容 “b” 被送入MDR内。
- ❷ 将MDR的内容传送至运算器的MQ (用 $(MDR) \rightarrow MQ$ 表示) 。
- ❸ CU向ALU发送乘法操作命令, 并把运算结果 ($a \times b$) 的高位存放在ACC中、低位存放在MQ中。

考点五：计算机运行原理

计算机运行原理



随着硬件技术的发展，内存都制成大规模集成电路芯片，
而将MAR和MDR集成到了CPU芯片中。

考点五：计算机运行原理

【例】存放当前指令的寄存器是？

- A.PC
- B.MDR
- C.IR
- D.MAR

【例】存放当前执行指令地址的寄存器是？

- A.PC
- B.MDR
- C.IR
- D.MAR

- 指令寄存器IR用来存放当前的指令
 - IR的内容来自存储器数据寄存器MDR。
 - 程序计数器PC用来存放当前欲执行指令的地址
 - PC与存储器地址寄存器MAR之间有一条直接通路。
 - PC自动形成下一条指令的地址（“自动加1”功能）
- 

考点五：计算机运行原理

【例】存储字是？

- A. 存放在一个存储单元中的二进制信息
- B. 一个存储单元可存储二进制信息的位数
- C. 存储体容量
- D. 存储单元的数量

【例】存储字长是？

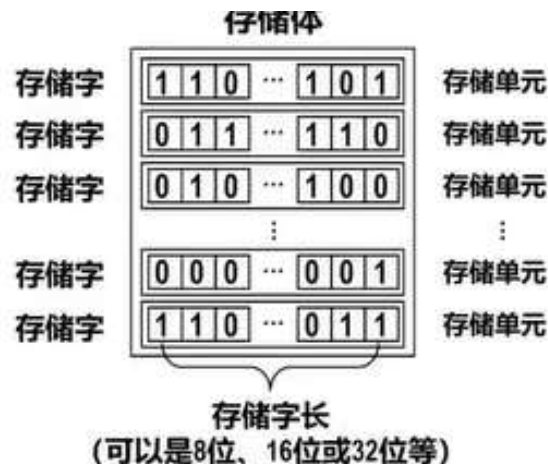
- A. 存放在一个存储单元中的二进制信息
- B. 一个存储单元可存储二进制信息的位数
- C. 存储体容量
- D. 存储单元的数量

■ **存储体**由很多个**存储单元**组成

☐ 每个**存储单元**由若干个**存储元件**组成

◇ 每个**存储元件**能存储**一位二进制数“0”或“1”**

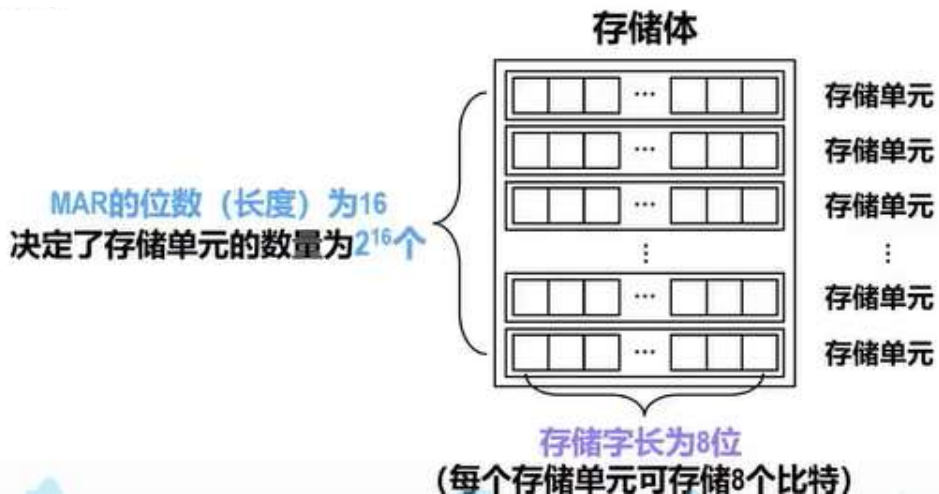
☐ 一个存储单元中可存储一串二进制信息，称这串二进制信息为一个**存储字**，这串**二进制信息的位数**称为**存储字长**（可以是8位、16位或32位等）。



考点五：计算机运行原理

【例】若存储字长为8位，MAR的位数（长度）是16位，则存储体的总容量是？

- A.128B
- B.64KB
- C.128KB
- D.256KB



存储体的总容量为：

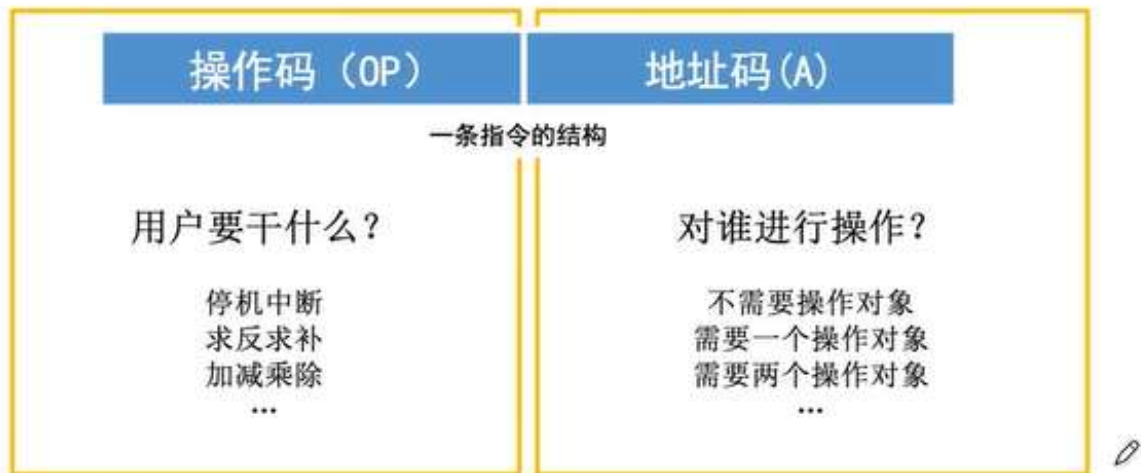
$$8\text{位} \times 2^{16}\text{个} = 2^{19}\text{位} = 2^{19}\text{比特} = 2^{19}\text{b}$$
$$2^{19}\text{b} \div 8 = 2^{16}\text{B} = 64\text{KB}$$

考点六：指令

指令格式

一条指令就是机器语言的一个语句，它是一组有意义的二进制代码。

一条指令通常要包括操作码字段和地址码字段两部分：



一条指令可能包含 0 个、1 个、2 个、3 个、4 个 地址码...

根据地址码数目不同，可以将指令分为 零地址指令、一地址指令、二地址指令...

考点六：指令

指令格式——按地址数量分类

零地址指令

OP

1. 不需要操作数，如空操作、停机、关中断等指令
2. 堆栈计算机，两个操作数隐含存放在栈顶和次栈顶，计算结果压回栈顶

一地址指令

OP

A₁

1. 只需要单操作数，如加1、减1、取反、求补等

指令含义：OP(A₁)→A₁， 完成一条指令需要3次访存：取指→读A₁→写A₁

2. 需要两个操作数，但其中一个操作数隐含在某个寄存器（如隐含在ACC）

指令含义：(ACC)OP(A₁)→ACC

考点六：指令

指令格式——按地址数量分类

二地址指令

OP

A₁（目的操作数）

A₂（源操作数）

常用于需要两个操作数的算术运算、逻辑运算相关指令

指令含义：(A₁)OP(A₂)→A₁

①

完成一条指令需要访存4次，取指→读A₁→读A₂→写A₁

三地址指令

OP

A₁

A₂

A₃（结果）

常用于需要两个操作数的算术运算、逻辑运算相关指令

指令含义：(A₁)OP(A₂)→A₃

完成一条指令需要访存4次，取指→读A₁→读A₂→写A₃

考点六：指令

指令格式——按地址数量分类

四地址指令

OP	A ₁	A ₂	A ₃ (结果)	A ₄ (下址)
----	----------------	----------------	---------------------	---------------------

指令含义：(A₁)OP(A₂)→A₃，A₄=下一条将要执行指令的地址

完成一条指令需要访存4次，取指→读A₁→读A₂→写A₃

正常情况下：取指令之后 PC+1，指向下一条指令

四地址指令：执行指令后，将PC的值修改为 A₄ 所指地址

考点六：指令

指令格式——按指令长度分类

指令字长：一条指令的总长度（可能会变）

机器字长：CPU进行一次整数运算所能处理的二进制数据的位数（通常和ALU直接相关）

存储字长：一个存储单元中的二进制代码位数（通常和MDR位数相同）

半字长指令、单字长指令、双字长指令 —— 指令长度是机器字长的多少倍

指令字长会影响取指令所需时间。如：机器字长=存储字长=16bit，则取一条双字长指令需要两次访存

定长指令字结构：指令系统中所有指令的长度都相等

变长指令字结构：指令系统中各种指令的长度不等

考点六：指令

指令格式——按操作码长度分类

定长操作码：指令系统中所有指令的操作码长度都相同
 n 位 $\rightarrow 2^n$ 条指令

控制器的译码电路设计简单，
但灵活性较低

可变长操作码：指令系统中各指令的操作码长度可变

控制器的译码电路设计复杂，
但灵活性较高

定长指令字结构+可变长操作码 $\rightarrow$ 扩展操作码指令格式

不同地址数的指令使用不同长度的操作码

考点六：指令

扩展操作码

指令字长为16位，每个地址码占4位：

前4位为基本操作码字段OP，另有3个4位长的地址字段A₁、A₂和A₃。

4位基本操作码若全部用于三地址指令，则有16条。

但至少须将1111留作扩展操作码之用，即三地址指令为15条；

1111 1111留作扩展操作码之用，二地址指令为15条；

1111 1111 1111留作扩展操作码之用，一地址指令为15条；

零地址指令为16条。

还有其他扩展操作码设计方法。

- 1) 不允许短码是长码的前缀，即短操作码不能与长操作码的前面部分的代码相同。
- 2) 各指令的操作码一定不能重复。

OP	A ₁	A ₂	A ₃
----	----------------	----------------	----------------

4位操作码	0000	A ₁	A ₂	A ₃	15条三地址指令
	0001	A ₁	A ₂	A ₃	
	⋮	⋮	⋮	⋮	
	1110	A ₁	A ₂	A ₃	

8位操作码	1111	0000	A ₂	A ₃	15条二地址指令
	1111	0001	A ₂	A ₃	
	⋮	⋮	⋮	⋮	
	1111	1110	A ₂	A ₃	

12位操作码	1111	1111	0000	A ₃	15条一地址指令
	1111	1111	0001	A ₃	
	⋮	⋮	⋮	⋮	
	1111	1111	1110	A ₃	

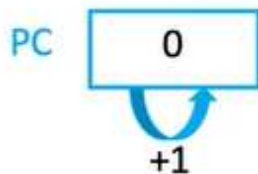
16位操作码	1111	1111	1111	0000	16条零地址指令
	1111	1111	1111	0001	
	⋮	⋮	⋮	⋮	
	1111	1111	1111	1111	

考点六：指令

指令寻址——顺序

指令寻址 下一条欲执行指令的地址 （始终由程序计数器PC给出）

$(PC) + 1 \rightarrow PC$



指令地址	操作码	地址码
0	LDA	1000
1	ADD	1001
2	DEC	1200
3	JMP	7
4	LDA	2000
5	SUB	2001
6	INC	
7	LDA	1100
8	...	

该系统采用定长指令字结构

指令字长=存储字长=16bit=2B

主存按字编址

考点六：指令

指令寻址——顺序

指令寻址 下一条 欲执行 指令 的 地址 （始终由程序计数器PC给出）

$(PC) + 1 \rightarrow PC$

$(PC) + 2 \rightarrow PC$

指令地址

0	0001001111101000
2	0011001111101001
4	0010010010110000
6	1001000000000111
8	0001011111010000
10	0100011111010001
12	0101011111010001
14	0001100111000100
...	...

该系统采用定长指令字结构

指令字长=存储字长=16bit=2B

主存按字节编址

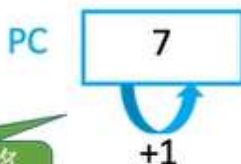
考点六：指令

指令寻址——跳跃

指令寻址 下一条欲执行指令的地址 （始终由程序计数器PC给出）

顺序寻址 (PC) + "1" → PC

跳跃寻址 由转移指令指出



执行转移指令，将PC值修改为7

顺序寻址

顺序寻址

顺序寻址

指令地址 操作码 地址码

0	LDA	1000
1	ADD	1001
2	DEC	1200
3	JMP	7
4	LDA	2000
5	SUB	2001
6	INC	
7	LDA	1100
8	...	

该系统采用定长指令字结构

指令字长=存储字长=16bit=2B

主存按字编址

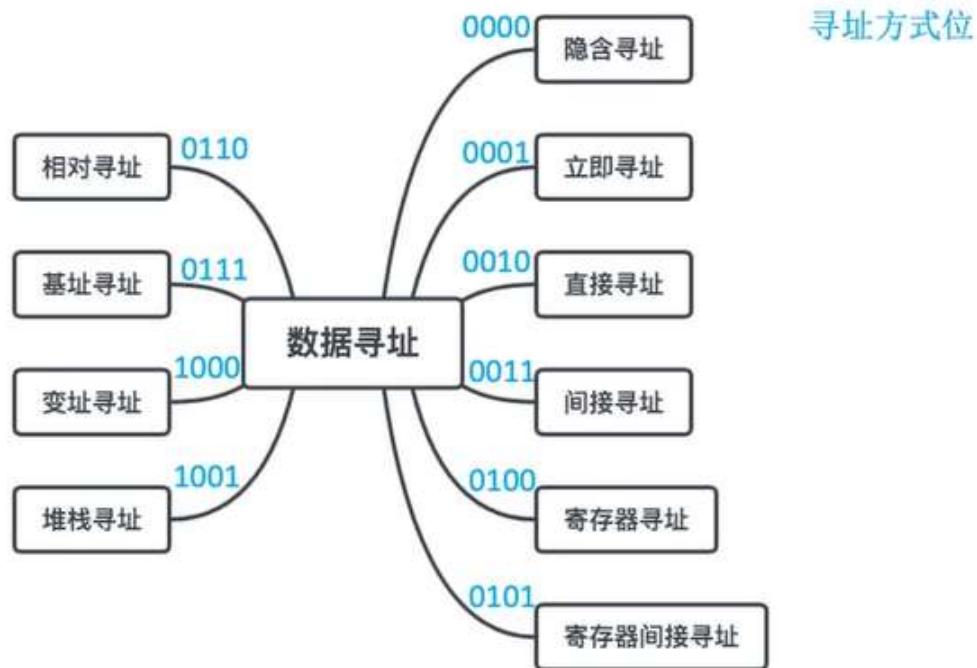
JMP: 无条件转移
把PC中的内容改成7

无条件转移指令，
类似C语言的 goto

考点六：指令

数据寻址

操作码 (OP)	寻址特征	形式地址 (A)
----------	------	----------



考点六：指令

数据寻址——直接寻址

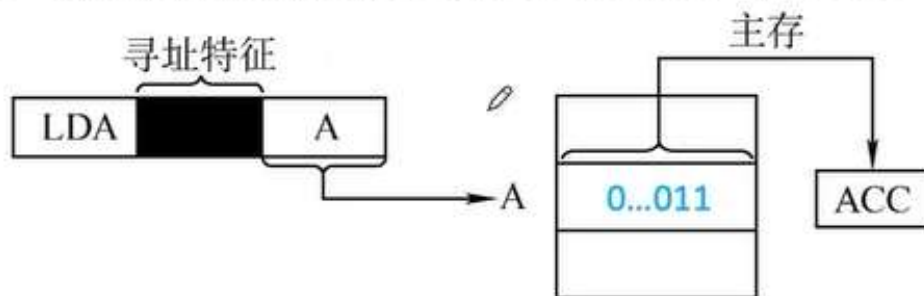
假设指令字长=机器字长=存储字长，操作数为3

一地址指令

操作码 (OP)

1001...0111

直接寻址：指令字中的形式地址A就是操作数的真实地址EA，即 $EA=A$ 。



一条指令的执行：
取指令 访存1次
执行指令 访存1次
暂不考虑存结果
共访存2次

优点：简单，指令执行阶段仅访问一次主存，
不需专门计算操作数的地址。

缺点：
A的位数决定了该指令操作数的寻址范围。
操作数的地址不易修改。

考点六：指令

数据寻址——间接寻址

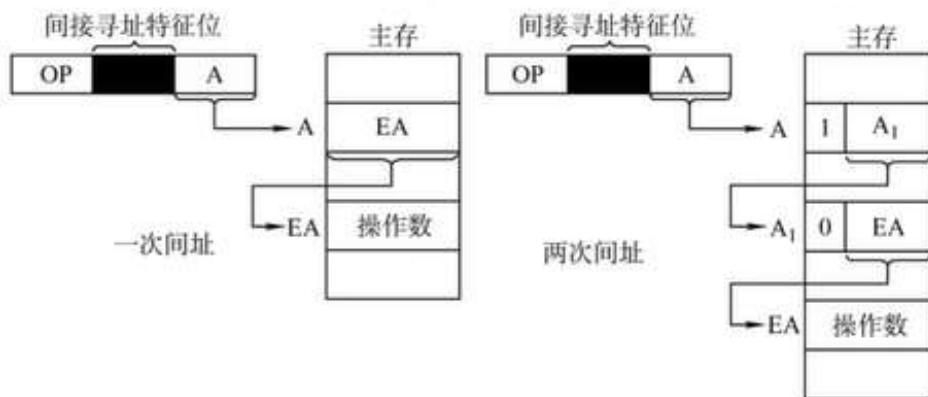
假设指令字长=机器字长=存储字长，操作数为3

一地址指令

操作码 (OP)

1001...0111

间接寻址：指令的地址字段给出的形式地址不是操作数的真正地址，而是操作数有效地址所在的存储单元的地址，也就是操作数地址的地址，即 $EA=(A)$ 。



优点：

可扩大寻址范围(有效地址EA的位数大于形式地址A的位数)。

便于编制程序(用间接寻址可以方便地完成子程序返回)。

缺点：

指令在执行阶段要多次访存(一次间址需两次访存，多次寻址需根据存储字的最高位确定几次访存)。

考点六：指令

数据寻址——寄存器寻址

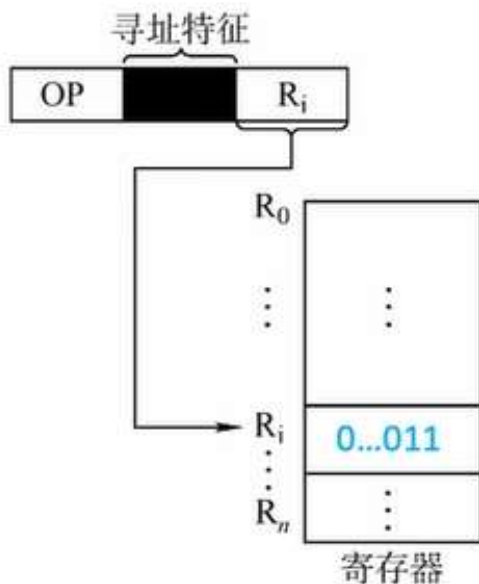
假设指令字长=机器字长=存储字长，操作数为3

一地址指令

操作码 (OP)

1001

寄存器寻址：在指令字中直接给出操作数所在的寄存器编号，即 $EA = R_i$ ，其操作数在由 R_i 所指的寄存器内。



一条指令的执行：
取指令 访存1次
执行指令 访存0次
暂不考虑存结果
共访存1次

优点：

指令在执行阶段不访问主存，只访问寄存器，
指令字短且执行速度快，支持向量/矩阵运算。

缺点：

寄存器价格昂贵，计算机中寄存器个数有限。

考点六：指令

数据寻址——寄存器间接寻址

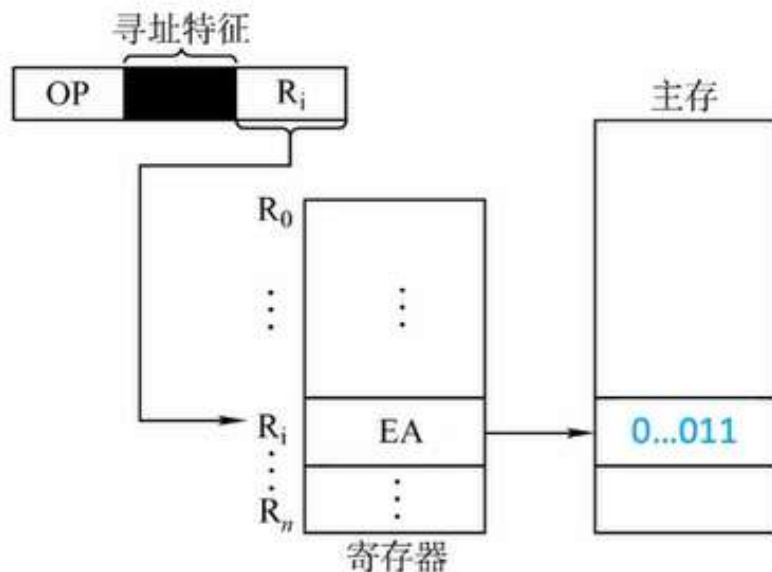
假设指令字长=机器字长=存储字长，操作数为3

一地址指令

操作码 (OP)

1001

寄存器间接寻址：寄存器 R_i 中给出的不是一个操作数，而是操作数所在主存单元的地址，即 $EA=(R_i)$ 。



一条指令的执行：

取指令 访存1次
执行指令 访存1次
暂不考虑存结果
共访存2次

特点：

与一般间接寻址相比速度更快，但指令的执行阶段需要访问主存(因为操作数在主存中)。

考点六：指令

数据寻址——立即寻址

假设指令字长=机器字长=存储字长，操作数为3

一地址指令

操作码 (OP)

#

0...011

立即寻址：形式地址A就是操作数本身，又称为立即数，一般采用补码形式。

#表示立即寻址特征。

一条指令的执行：

取指令 访存1次

执行指令 访存0次

暂不考虑存结果

共访存1次

优点：指令执行阶段不访问主存，指令执行时间最短

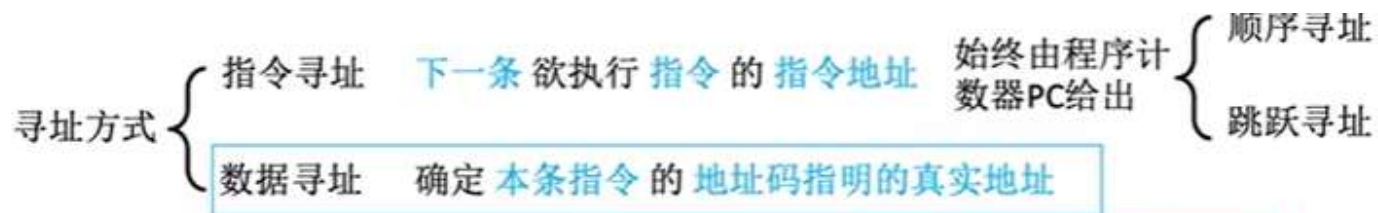
缺点：

A的位数限制了立即数的范围。

如A的位数为n，且立即数采用补码时，可表示的数据范围为 $-2^{n-1} \sim 2^{n-1} - 1$

考点六：指令

数据寻址——偏移寻址



0	LDA	1000
1	ADD	1001
2	DEC	1200
3	JMP	7
4	LDA	2000
5	SUB	2001
6	INC	
7	LDA	1100
8	...	

起始 →

100	LDA	1000
101	ADD	1001
102	DEC	1200
103	JMP	7
104	LDA	2000
105	SUB	2001
106	INC	
107	LDA	1100
108	...	

以某个地址作为起点
形式地址视为“偏移量”

PC →

100	LDA	1000
101	ADD	1001
102	DEC	1200
103	JMP	3
104	LDA	2000
105	SUB	2001
106	INC	
107	LDA	1100
108	...	

考点六：指令

数据寻址——偏移寻址



区别在于偏移的“起点”不一样

不一样
我们不一样



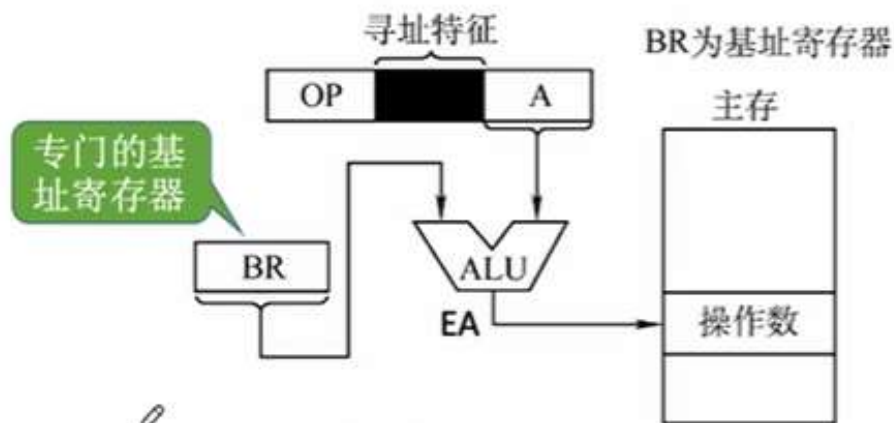
基址寻址：以程序的起始存放地址作为“起点”
变址寻址：程序员自己决定从哪里作为“起点”
相对寻址：以程序计数器PC所指地址作为“起点”

考点六：指令

数据寻址——偏移寻址——基址

基址寻址：将CPU中基址寄存器（BR）的内容加上指令格式中的形式地址A，而形成操作数的有效地址，即 $EA = (BR) + A$ 。

注：BR——base address register
EA——effective address



(a) 采用专用寄存器BR作为基址寄存器

注：基址寄存器是面向操作系统的，其内容由操作系统或管理程序确定。在程序执行过程中，基址寄存器的内容不变（作为基地址），形式地址可变（作为偏移量）。

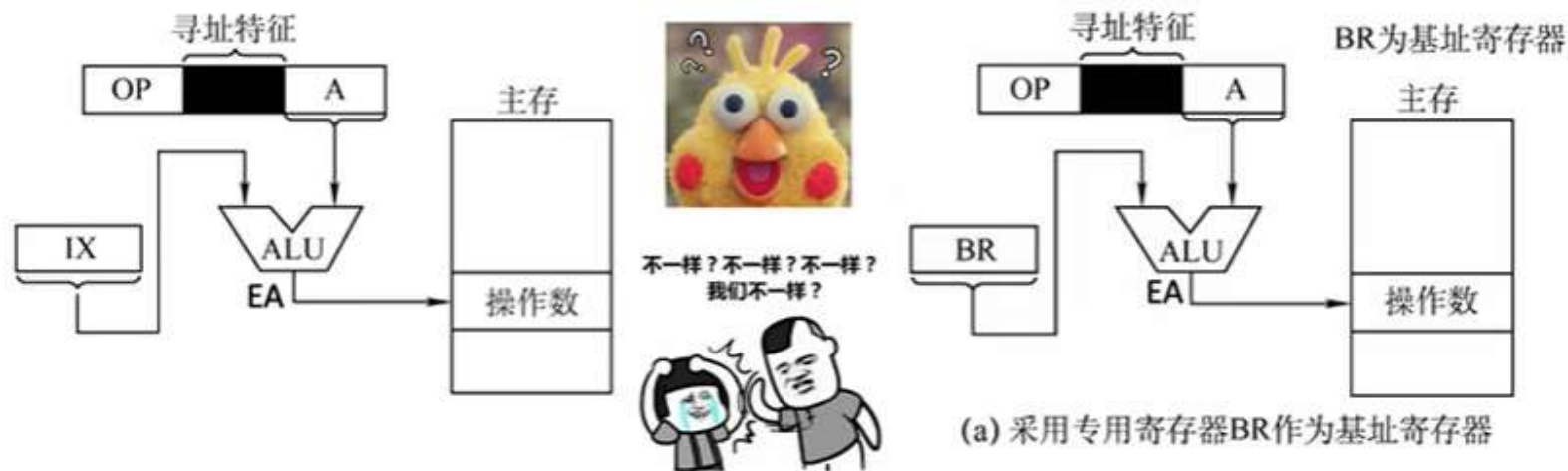
当采用通用寄存器作为基址寄存器时，可由用户决定哪个寄存器作为基址寄存器，但其内容仍由操作系统确定。

优点：可扩大寻址范围（基址寄存器的位数大于形式地址A的位数）；用户不必考虑自己的程序存于主存的哪一空间区域，故有利于多道程序设计，以及可用于编制浮动程序（整个程序在内存里边的浮动）。

考点六：指令

数据寻址——偏移寻址——变址

变址寻址：有效地址EA等于指令字中的形式地址A与变址寄存器IX的内容相加之和，即 $EA = (IX) + A$ ，其中IX可为变址寄存器（专用），也可用通用寄存器作为变址寄存器。



注：变址寄存器是面向用户的，在程序执行过程中，变址寄存器的内容可由用户改变（IX作为偏移量），形式地址A不变（作为基地址）。

基址寻址中，BR保持不变作为基地址，A作为偏移量

考点六：指令

CISC/RISC

对比项目 \ 类别	CISC	RISC
指令系统	复杂，庞大	简单，精简
指令数目	一般大于200条	一般小于100条
指令字长	不固定	定长
可访存指令	不加限制	只有Load/Store指令
各种指令执行时间	相差较大	绝大多数在一个周期内完成
各种指令使用频度	相差很大	都比较常用
通用寄存器数量	较少	多
目标代码	难以用优化编译生成高效的目标代码程序	采用优化的编译程序，生成代码较为高效
控制方式	绝大多数为微程序控制	绝大多数为组合逻辑控制
指令流水线	可以通过一定方式实现	必须实现

考点六：指令

【例】下列数据寻址中，最适合按照下标顺序访问一维数组元素的是？

- A.相对寻址
- B.寄存器寻址
- C.直接寻址
- D.变址寻址

考点六：指令

【例】下列数据寻址中，最适合按照下标顺序访问一维数组元素的是？

- A.相对寻址
- B.寄存器寻址
- C.直接寻址
- D.变址寻址

【解析】在变址操作时，将计算机指令中的地址与变址寄存器中的地址相加，得到有效地址，指令提供数组首地址，由变址寄存器来定位数据中的各元素。所以它最适合按下标顺序访问一维数组元素，选D。相对寻址以PC为基地址，以指令中的地址为偏移量确定有效地址。寄存器寻址则在指令中指出需要使用的寄存器。直接寻址在指令的地址字段直接指出操作数的有效地址。

考点七：指令流水线

一条指令的执行过程可以分成多个阶段（或过程）。
根据计算机的不同，具体的分法也不同。

取指	分析	执行
----	----	----

特点：每个阶段用到的硬件不一样。

取指：根据PC内容访问主存储器，取出一条指令送到IR中。

分析：对指令操作码进行译码，按照给定的寻址方式和地址字段中的内容形成操作数的有效地址EA，并从有效地址EA中取出操作数。

执行：根据操作码字段，完成指令规定的功能，即把运算结果写到通用寄存器或主存中。

设取指、分析、执行3个阶段的时间都相等，用 t 表示，按以下几种执行方式分析 n 条指令的执行时间：

1. 顺序执行方式 总耗时 $T = n \times 3t = 3nt$



传统冯·诺依曼机采用顺序执行方式，又称串行执行方式。

优点：控制简单，硬件代价小。

缺点：执行指令的速度较慢，在任何时刻，处理机中只有一条指令在执行，各功能部件的利用率很低。

考点七：指令流水线

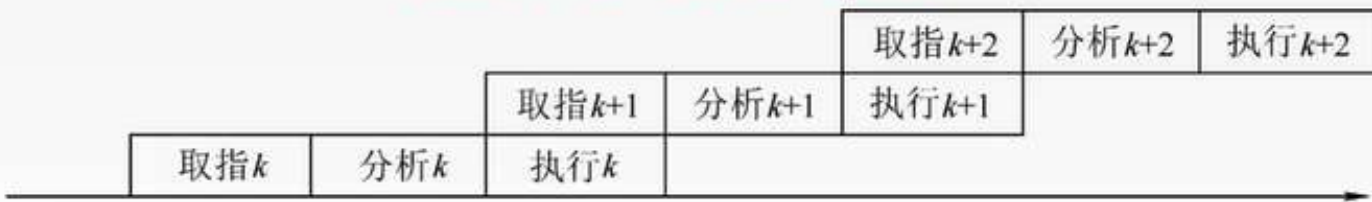
1. 顺序执行方式 总耗时 $T = n \times 3t = 3nt$



传统冯·诺依曼机采用顺序执行方式，又称串行执行方式。优点：控制简单，硬件代价小。

缺点：执行指令的速度较慢，在任何时刻，处理机中只有一条指令在执行，各功能部件的利用率很低。

2. 一次重叠执行方式 总耗时 $T = 3t + (n-1) \times 2t = (1+2n)t$



优点：程序的执行时间缩短了1/3，各功能部件的利用率明显提高。

缺点：需要付出硬件上较大开销的代价，控制过程也比顺序执行复杂了。

3. 二次重叠执行方式 总耗时 $T = 3t + (n-1) \times t = (2+n)t$



与顺序执行方式相比，指令的执行时间缩短近2/3。这是一种理想的指令执行方式，在正常情况下，处理机中同时有3条指令在执行。

考点七：指令流水线

流水线性能指标——吞吐率

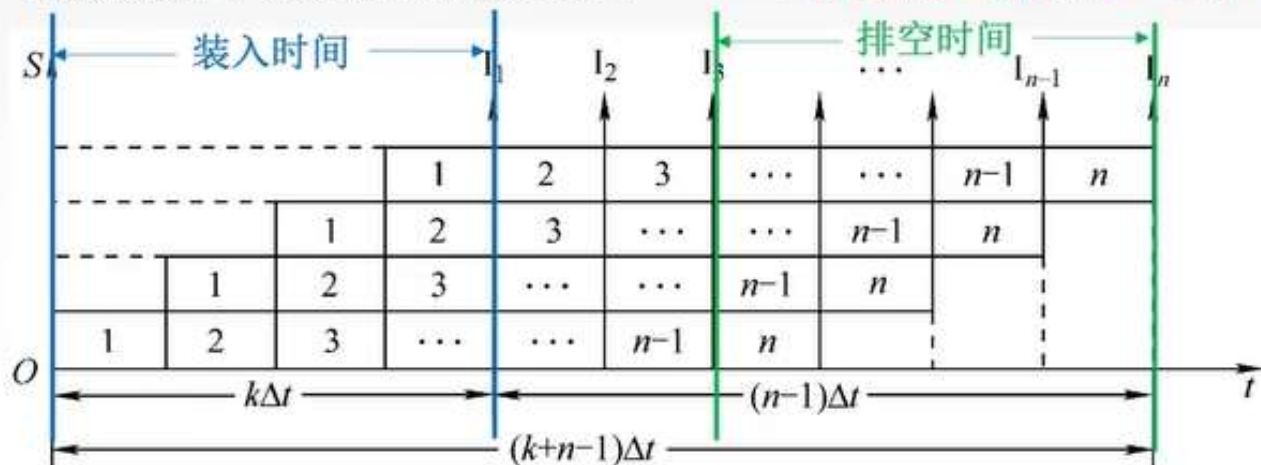
1. **吞吐率** 吞吐率是指在单位时间内流水线所完成的任务数量，或是输出结果的数量。

设任务数为 n ；处理完成 n 个任务所用的时间为 T_k

则计算流水线吞吐率（TP）的最基本的公式为 $TP = \frac{n}{T_k}$

理想情况下，流水线的时空图如下：

当连续输入的任务 $n \rightarrow \infty$ 时，得最大吞吐率为 $TP_{\max} = 1/\Delta t$ 。



$$T_k = (k+n-1) \Delta t$$

流水线的实际吞吐率为

$$TP = \frac{n}{(k+n-1)\Delta t}$$

一条指令的执行分为 k 个阶段，每个阶段耗时 Δt ，一般取 Δt = 一个时钟周期

考点七：指令流水线

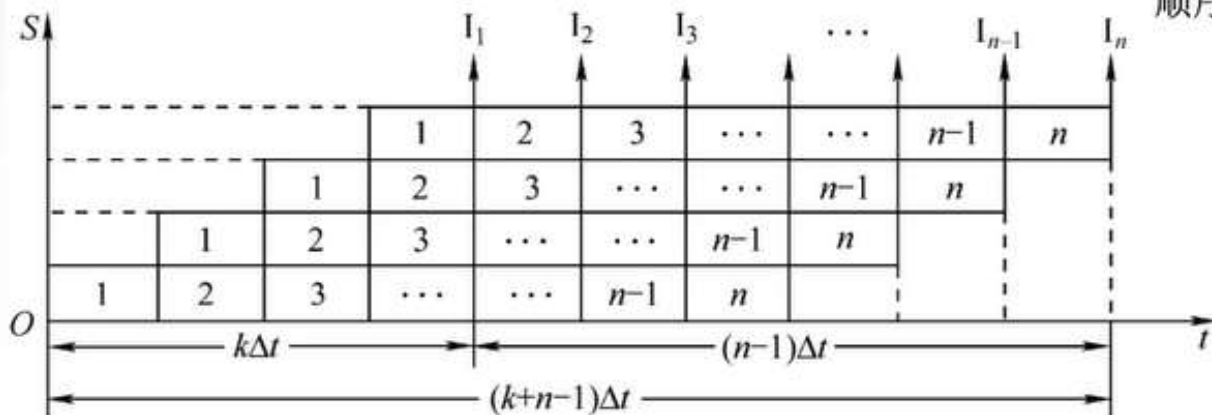
流水线性能指标——加速比

2. 加速比 完成同样一批任务，不使用流水线所用的时间与使用流水线所用的时间之比。

设 T_0 表示不使用流水线时的执行时间，即顺序执行所用的时间； T_k 表示使用流水线时的执行时间

则计算流水线加速比(S)的基本公式为 $S = \frac{T_0}{T_k}$ 当连续输入的任务 $n \rightarrow \infty$ 时，最大加速比为 $S_{\max}=k$ 。

理想情况下，流水线的时空图如下：



单独完成一个任务耗时为 $k \Delta t$ ，则
顺序完成 n 个任务耗时 $T_0 = nk \Delta t$

$$T_k = (k+n-1) \Delta t$$

实际加速比为

$$S = \frac{kn\Delta t}{(k+n-1)\Delta t} = \frac{kn}{k+n-1}$$

一条指令的执行分为 k 个阶段，每个阶段耗时 Δt ，一般取 Δt = 一个时钟周期

考点七：指令流水线

流水线性能指标——效率

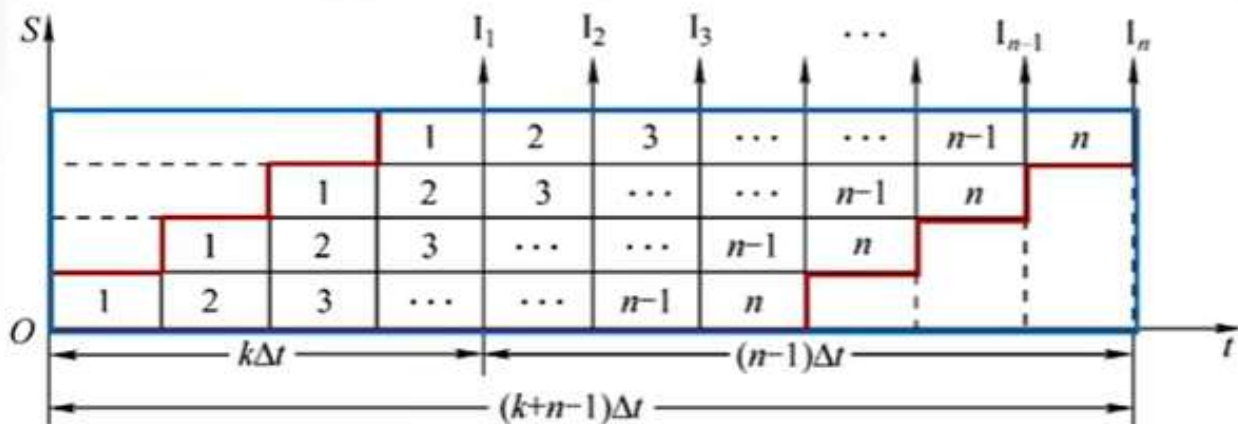
3. 效率

流水线的设备利用率称为流水线的效率。

在时空图上，流水线的效率定义为完成 n 个任务占用的时空区有效面积与 n 个任务所用的时间与 k 个流水段所围成的时空区总面积之比。

则流水线效率 (E) 的一般公式为 $E = \frac{n \text{个任务占用} k \text{时空区有效面积}}{n \text{个任务所用的时间与} k \text{个流水段所围成的时空区总面积}} = \frac{T_0}{kT_k}$

理想情况下，流水线的时空图如下：

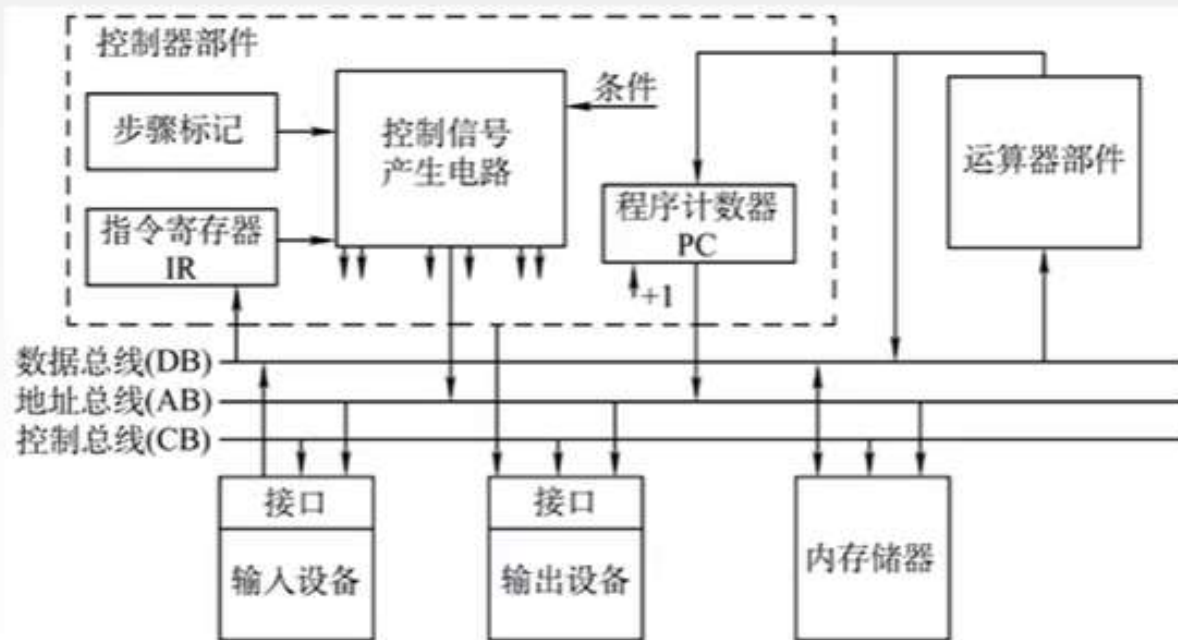


当连续输入的任务 $n \rightarrow \infty$ 时，最高效率为 $E_{\max}=1$ 。

一条指令的执行分为 k 个阶段，每个阶段耗时 Δt ，一般取 $\Delta t = \text{一个时钟周期}$

考点八：总线

总线是一组能为多个部件**分时共享**的公共信息传送线路。



共享是指总线上可以挂接多个部件，各个部件之间互相交换的信息都可以通过这组线路**分时共享**。

分时是指同一时刻只允许有一个部件向总线发送信息，如果系统中有多多个部件，则它们只能**分时**地向总线发送信息。

为什么要用总线？

早期计算机外部设备少时大多采用分散连接方式，不易实现随时增减外部设备。为了更好地解决I/O设备和主机之间连接的灵活性问题，计算机的结构从分散连接发展为总线连接。

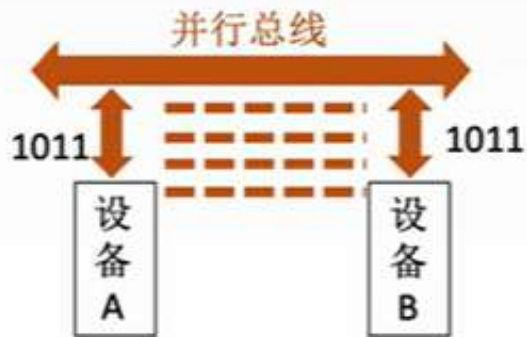
考点八：总线

总线分类——串/并行



优点：只需要一条传输线，成本低廉，广泛应用于长距离传输；应用于计算机内部时，可以节省布线空间。

缺点：在数据发送和接收的时候要进行拆卸和装配，要考虑串行-并行转换的问题。



优点：总线的逻辑时序比较简单，电路实现起来比较容易。

缺点：信号线数量多，占用更多的布线空间；远距离传输成本高昂；由于工作频率较高时，并行的信号线之间会产生严重干扰，对每条线等长的要求也越高，所以无法持续提升工作频率。

考点八：总线

总线分类——功能

数据通路表示的是数据流经的路径
数据总线是承载的媒介

1. 片内总线

片内总线是芯片内部的总线。

它是CPU芯片内部寄存器与寄存器之间、寄存器与ALU之间的公共连接线。

2. 系统总线

系统总线是计算机系统内各功能部件（CPU、主存、I/O接口）之间相互连接的总线。

按系统总线传输信息内容的不同，又可分为3类：数据总线、地址总线和控制总线。

1) 数据总线用来传输各功能部件之间的数据信息，它是双向传输总线，其位数与机器字长、存储字长有关。

2) 地址总线用来指出数据总线上的源数据或目的数据所在的主存单元或I/O端口的地址，它是单向传输总线，地址总线的位数与主存地址空间的大小有关。

3) 控制总线传输的是控制信息，包括CPU送出的控制命令和主存（或外设）返回CPU的反馈信号。

3. 通信总线

通信总线是用于计算机系统之间或计算机系统与其他系统（如远程通信设备、测试设备）之间信息传送的总线，通信总线也称为外部总线。

考点八：总线

总线性能指标

1. 总线的传输周期(总线周期)

一次总线操作所需的时间（包括申请阶段、寻址阶段、传输阶段和结束阶段），通常由若干个总线时钟周期构成。

2. 总线时钟周期

即机器的时钟周期。计算机有一个统一的时钟，以控制整个计算机的各个部件，总线也要受此时钟的控制。

3. 总线的工作频率

总线上各种操作的频率，为总线周期的倒数。实际上指一秒内传送几次数据。

4. 总线的时钟频率

即机器的时钟频率，为时钟周期的倒数。实际上指一秒内有多少个时钟周期。

5. 总线宽度

又称为总线位宽，它是总线上同时能够传输的数据位数，通常是指数据总线的根数，如32根称为32位（bit）总线。

6. 总线带宽

可理解为总线的数据传输率，即单位时间内总线上可传输数据的位数，通常用每秒钟传送信息的字节数来衡量，单位可用字节/秒（B/s）表示。

总线带宽 = 总线工作频率 × 总线宽度（bit/s） = 总线工作频率 × (总线宽度/8)（B/s）

7. 总线复用

总线复用是指一种信号线在不同的时间传输不同的信息。可以使用较少的线传输更多的信息，从而节省了空间和成本。

8. 信号线数

地址总线、数据总线和控制总线3种总线数的总和称为信号线数。

考点八：总线

总线仲裁



同一时刻只能有一个设备控制总线传输操作，可以有一个或多个设备从总线接收数据。

将总线上所连接的各类设备按其对总线有无控制功能分为：

主设备：获得总线控制权的设备。

从设备：被主设备访问的设备，只能响应从主设备发来的各种总线命令。

为什么要仲裁？

总线作为一种共享设备，不可避免地会出现同一时刻有多个主设备竞争总线控制权的问题。

总线仲裁的定义：

多个主设备同时竞争总线控制权时，以某种方式选择一个主设备优先获得总线控制权称为总线仲裁。

总线仲裁分类：

集中仲裁方式 链式查询方式、计数器定时查询方式、独立请求方式

分布仲裁方式

考点八：总线

集中总线仲裁——菊花链 (Daisy Chain)

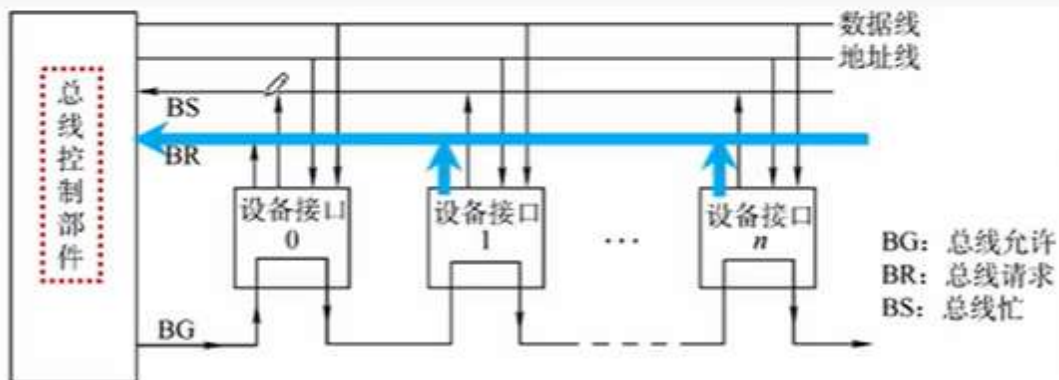
工作流程：

1. 主设备发出请求信号；
2. 若多个主设备同时要使用总线，则由总线控制器的判优、仲裁逻辑按一定的优先等级顺序确定哪个主设备能使用总线；
3. 获得总线使用权的主设备开始传送数据。

链式查询方式

计数器查询方式

独立请求方式



考点八：总线

集中总线仲裁——菊花链 (Daisy Chain)

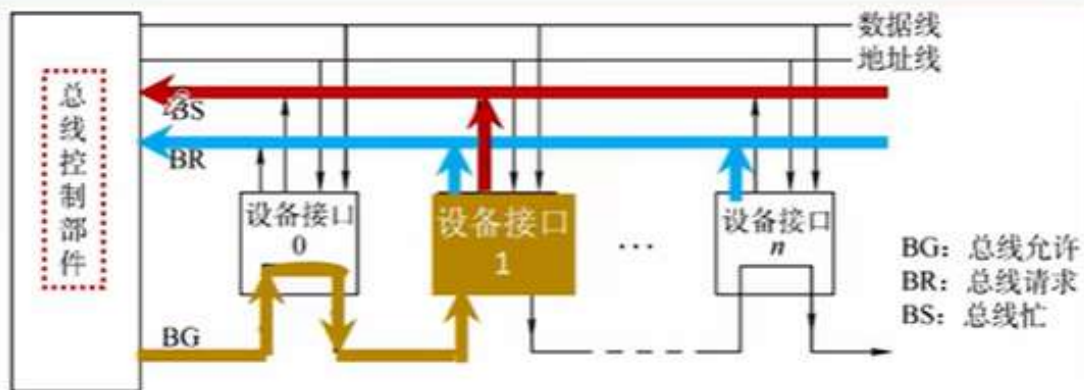
工作流程：

1. 主设备发出请求信号；
2. 若多个主设备同时要使用总线，则由总线控制器的判优、仲裁逻辑按一定的优先等级顺序确定哪个主设备能使用总线；
3. 获得总线使用权的主设备开始传送数据。

链式查询方式

计数器查询方式

独立请求方式



考点八：总线

集中总线仲裁——菊花链 (Daisy Chain)

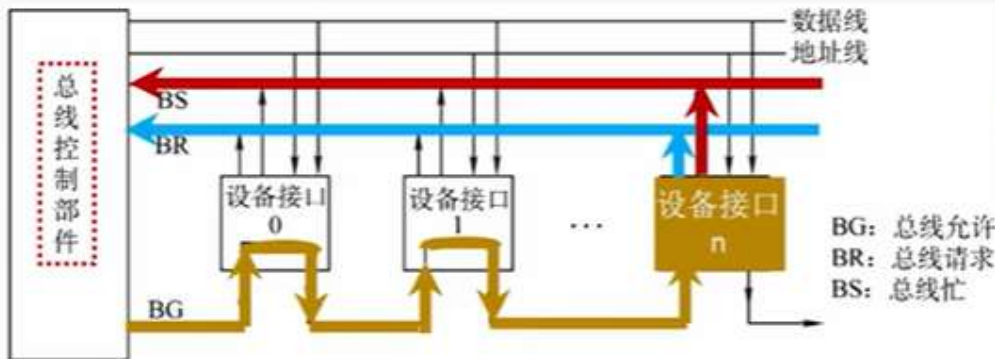
工作流程：

1. 主设备发出请求信号；
2. 若多个主设备同时要使用总线，则由总线控制器的判优、仲裁逻辑按一定的优先等级顺序确定哪个主设备能使用总线；
3. 获得总线使用权的主设备开始传送数据。

链式查询方式

计数器查询方式

独立请求方式



“总线忙”信号的建立者是获得总线控制权的设备

优先级：
离总线控制器越近的部件，其优先级越高；
离总线控制器越远的部件，其优先级越低。

优点：链式查询方式优先级固定。

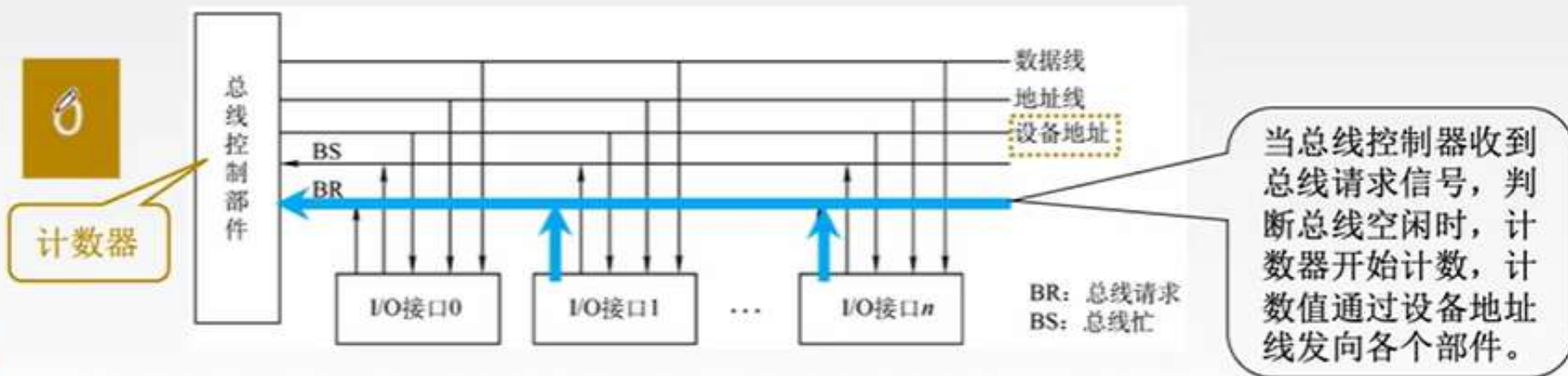
只需很少几根控制线就能按一定优先次序实现总线控制，结构简单，扩充容易。

缺点：对硬件电路的故障敏感，并且优先级不能改变。

当优先级高的部件频繁请求使用总线时，会使优先级较低的部件长期不能使用总线。

考点八：总线

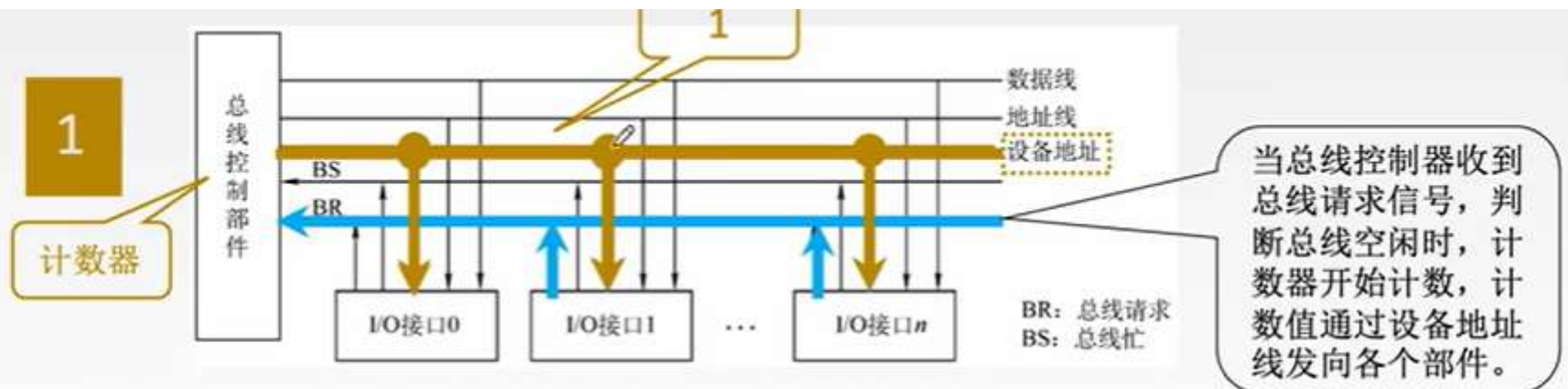
集中总线仲裁——计数器



结构特点：用一个计数器控制总线使用权，相对链式查询方式多了一组设备地址线，少了一根总线响应线BG；它仍共用一根总线请求线。

考点八：总线

集中总线仲裁——计数器



结构特点：用一个计数器控制总线使用权，相对链式查询方式多了一组设备地址线，少了一根总线响应线BG；它仍共用一根总线请求线。

优点：

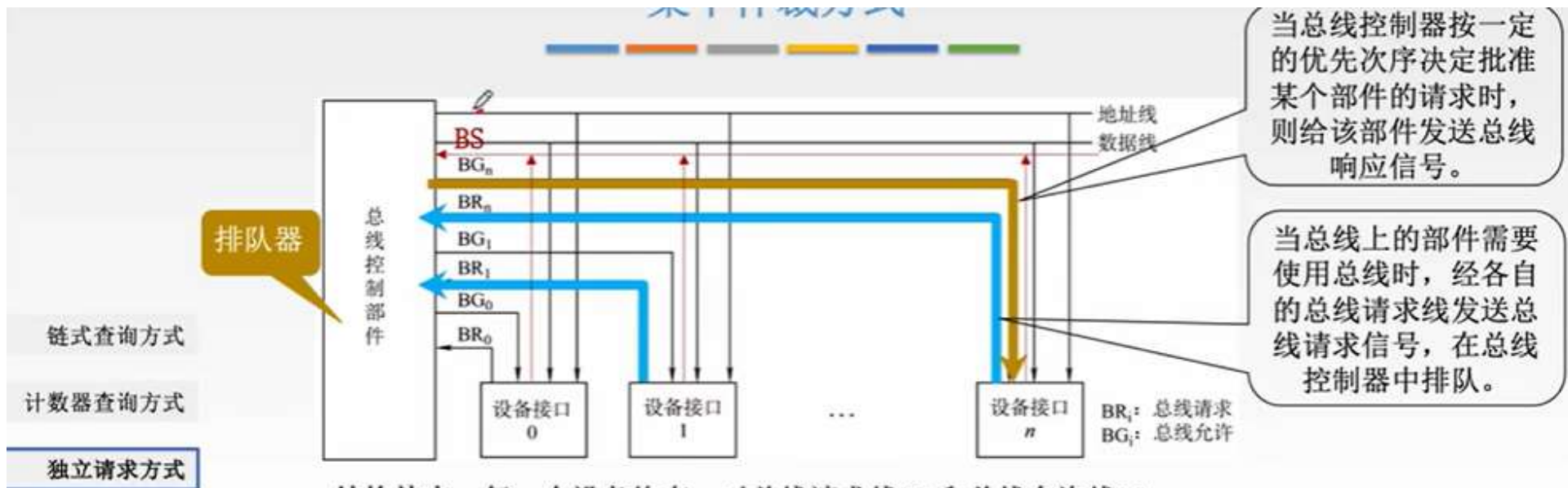
1. 计数初始值可以改变优先次序
 - 计数每次从“0”开始，设备的优先级就按顺序排列，固定不变；
 - 计数从上一轮的终点开始，此时设备使用总线的优先级相等；
 - 计数器的初值还可以由程序设置
2. 对电路的故障没有链式敏感

缺点：

1. 增加了控制线数
 - 若设备有 n 个，则需 $\lceil \log_2 n \rceil + 2$ 条控制线
2. 控制相对比链式查询相对复杂

考点八：总线

集中总线仲裁——独立请求



结构特点：每一个设备均有一对总线请求线BR_i和总线允许线BG_i。

优点：

1. 响应速度快，总线允许信号BG直接从控制器发送到有关设备，不必在设备间传递或者查询。
2. 对优先次序的控制相当灵活。

缺点：

1. 控制线数量多

-若设备有n个，则需要2n+1条控制线。

其中+1为BS线，用于设备向总线控制部件反馈是否正在使用总线。

2. 总线的控制逻辑更加复杂

考点八：总线

集中总线仲裁

仲裁方式 对比项目	链式查询	计数器定时查询	独立请求
控制线数	3 总线请求：1 总线允许：1 总线忙：1	$\lceil \log_2 n \rceil + 2$ 总线请求：1 总线允许： $\lceil \log_2 n \rceil$ 总线忙：1	$2n+1$ 总线请求：n 总线允许：n 总线忙：1
优点	优先级固定 结构简单，扩充容易	优先级较灵活	响应速度快 优先级灵活
缺点	对电路故障敏感 优先级不灵活	控制线较多 控制相对复杂	控制线多 控制复杂

考点八：总线

分布总线仲裁

特点：不需要中央仲裁器，每个潜在的主模块都有自己的仲裁器和仲裁号，多个仲裁器竞争使用总线。

当设备有总线请求时，它们就把各自唯一的仲裁号发送到共享的仲裁总线上；

每个仲裁器将从仲裁总线上得到的仲裁号与自己的仲裁号进行比较；

如果仲裁总线上的号优先级高，则它的总线请求不予响应，并撤销它的仲裁号；

最后，获胜者的仲裁号保留在仲裁总线上。